AI Personal Assistant with Autogen

A multi-agent personal assistant system built with AutoGen AgentChat that provides email, calendar, weather, and web search capabilities through a FastAPI interface with optional Slack integration.

Features

- **Multi-Agent System**: Specialized agents for email, calendar, weather, and web search
- **Email Management**: Gmail integration for reading, searching, drafting, and sending emails
- **Calendar Management**: Google Calendar integration for event scheduling and management
- **Weather Information**: Real-time weather data and forecasts
- **Web Search**: Tavily-powered search for web information, news, and research
- **Slack Integration**: Optional Slack bot for team collaboration with threaded responses
- **FastAPI Backend**: Modern, fast web API with automatic documentation

Project Structure

```
personal-assistant/
├── source/
│   ├──__init__.py
│   ├── personal_agents.py      # Main orchestrator and agent definitions
│   ├── email_tools.py          # Gmail integration tools
│   ├── configurations.py       # Configuration management
│   ├── calendar_tools.py       # Google Calendar tools
│   ├── weather_tools.py        # Weather API tools
│   ├── search_tools.py         # Tavily search integration
│   ├── google_utils.py         # Google API utilities
│   ├── prompts.py              # System prompts for all agents
│   └── log_records.py          # Logging configuration
├── slack_app.py                # Slack integration server
├── requirements.txt            # Python dependencies
├── credentials.json            # Google API credentials (you create this)
├── token.json                  # OAuth token (auto-generated)
└── logs/                       # Log files directory
```

Setup

1. Create Virtual Environment (Python 3.12)

```
# Create virtual environment
python -m venv vn_autogen

# Activate virtual environment
# On Windows:
vn_autogen\Scripts\activate
```

```
# On macOS/Linux:
source vn_autogen/bin/activate
```

2. Install Dependencies

```
pip install -r requirements.txt
```

3. Environment Configuration

Edit `configurations.py` with your configuration:

```
# Required
OPENAI_API_KEY = "your_openai_api_key_here"
TAVILY_SEARCH_KEY = "your_tavily_api_key_here"

# Optional - Slack Integration
SLACK_BOT_TOKEN = "xoxb-your-slack-bot-token"
SLACK_SIGNING_SECRET = "your_slack_signing_secret"

# Optional - Customization
OPENAI_MODEL = "gpt-4o"
OPENAI_TEMPERATURE = 1
MAX_RESULTS = 5
DEFAULT_TIMEZONE = "Asia/Kolkata"
```

4. Google API Setup

1. Create a project in Google Cloud Console
2. Enable Gmail API and Google Calendar API
3. Create credentials (OAuth 2.0 Client IDs)
4. Download the credentials file as `gmail-creds.json`
5. Place the file in the project root directory

5. Tavily API Setup

1. Sign up at Tavily
2. Get your API key from the dashboard
3. Add it to your `configurations.py` file

6. Run Initial OAuth Flow

On first run, the application will open a browser window for Google OAuth authorization. This creates a `token.json` file for future authentication.

Running the Application

```
# Activate virtual environment first
source vn_autogen/bin/activate  # macOS/Linux
# OR
vn_autogen\Scripts\activate     # Windows

# Run the Slack API server
python slack_app.py

# Or run with uvicorn
uvicorn slack_app:app --host 0.0.0.0 --port 8000
```

API Endpoints

- **POST /slack/events**: Slack event handler (app mentions)

Agent Capabilities

Email Agent

- Search emails by sender, subject, or content
- Read specific emails and threads
- Draft and send new emails
- Manage email organization
- Time-aware operations with current datetime

Calendar Agent

- Create, update, and delete events with Meet integration
- Schedule recurring meetings
- Find free time slots
- Respond to meeting invitations
- Bulk event creation
- Time-aware scheduling with current datetime

Weather Agent

- Current weather conditions for any location
- Weather forecasts (hourly/daily/tomorrow)
- Rain probability predictions
- Location geocoding

Search Agent

- General web search with Tavily
- Comprehensive research with detailed analysis
- Uses current datetime for temporal queries

Slack Integration

Setup Slack App

1. Create a Slack app at api.slack.com
2. Configure OAuth scopes:
    - `app_mentions:read`
    - `chat:write`
    - `commands`
3. Enable event subscriptions and point to your server
4. Set your Slack credentials in `configurations.py`
5. Install the app to your workspace

Example Slack Usage

```
@assistant What's the weather in Delhi?
@assistant Schedule a meeting tomorrow at 3 PM
@assistant Search for latest news about artificial intelligence
@assistant Find my emails from john@example.com
```

Configuration

All configuration is managed through the `configurations.py` file. Key settings include:

- **OpenAI Configuration**: API key, model selection, temperature
- **Tavily Configuration**: Search API key and result limits
- **Slack Configuration**: Bot token and signing secret
- **Google API Configuration**: Credentials file paths
- **Timezone Configuration**: Default timezone for all operations

Development

Adding New Tools

1. Create tool methods in the relevant class (e.g., `SearchTools`)
2. Add tools to the `as_function_tools()` method
3. Update the agent's system prompt in `prompts.py`

Adding New Agents

1. Create the agent in `personal_agents.py`
2. Add agent to the orchestrator's agent list
3. Create corresponding tools and prompts

Testing Individual Components

```
# Test search tools
python -m source.search_tools.py
```

```
# Test weather tools
python -m source.weather_tools.py

# Test calendar tools
python -m source.calendar_tools.py

# Test full orchestrator
python -m source.personal_agents.py
```

Logging

All operations are logged to:

- Console output for real-time monitoring
- `logs/personal_assistant.log` for persistent logging
- Structured logging with timestamps and log levels

Troubleshooting

Common Issues

1. **Virtual Environment Issues**:

   ```
   # Ensure virtual environment is activated
   which python  # Should point to your venv
   ```

2. **Google OAuth Issues**:
   - Ensure `credentials.json` is properly configured
   - Check Google Cloud Console API quotas
   - Verify OAuth consent screen is configured

3. **Slack Verification Failed**:
   - Check signing secret and bot token
   - In your Slack App settings: Go to Event Subscriptions Set Request URL to: https://abc123.ngrok.io/slack/events Slack will verify the URL

4. **Tavily API Errors**:
   - Verify API key is correct
   - Check rate limits and usage quotas
   - Ensure proper internet connectivity

5. **OpenAI API Errors**:
   - Verify API key and check rate limits
   - Monitor token usage for large requests
   - Check model availability

6. **Import Errors**:
    - Ensure virtual environment is activated
    - Install all dependencies: `pip install -r requirements.txt`
    - Check Python version compatibility